

Phase 3 comparison with the Databricks coding-agent benchmark

Date: 2026-07-13

Scope: Comparison of our Phase 3 Claude Code backend benchmark against the Databricks coding-agent benchmark article.

Purpose

Databricks published an internal coding-agent benchmark focused on real engineering tasks from its multi-million-line codebase. Their report emphasizes cost versus performance, capability tiers, the difference between token price and task-level cost, and the effect of the harness on model efficiency.

Our Phase 3 benchmark answers a related but narrower question: with Claude Code fixed as the agent harness, which backend/model routes perform best on Terminal-Bench/Harbor, and how much cost is consumed by successful, failed, incomplete, or operationally unclean attempts?

This note compares the two reports and highlights where the findings agree, where they differ, and what our results add.

High-level comparison

Dimension	Databricks benchmark	Our Phase 3 benchmark
Workload	Internal PR-derived coding tasks from a multi-million-line production codebase	Terminal-Bench 2.0 tasks run through Harbor
Harness setup	Multiple agent harnesses/tools compared, including native and simpler harnesses	Claude Code fixed as the harness; backend/model route varies
Main chart	Cost versus performance	Adjusted cost versus pass rate
Correctness signal	Held-out tests, no LLM judge	Terminal-Bench verifier/reward, no LLM judge
Main economic metric	Cost per task	Adjusted known cost per benchmark attempt, cost per clean success
Added accounting layer	Task-level cost emphasis	Recorded cost, adjusted known cost, accounting gap, unclean spend, token waste
Key operational lesson	Use a mix of models and harnesses; do not rely on token price alone	Use a routing strategy; track failed/unclean spend and token waste as first-class outcomes

Finding 1: both benchmarks show a diverse frontier

Databricks found that the Pareto frontier for coding tasks includes models from OpenAI, Anthropic, and open-source families. Our adjusted cost/performance frontier is similarly diverse:

- router-glm-5.1
- router-glm-5.2
- router-anthropic-fable-5
- router-deepseek-flash
- router-gpt-5.5

This is an important agreement. Neither benchmark supports a one-model answer. The practical conclusion is routing: different models occupy different cost/performance regions, and a deployment strategy should choose among them based on task difficulty, cost tolerance, and operational risk.

Finding 2: open or non-frontier-provider models are economically important

Databricks highlighted GLM 5.2 as evidence that open models are now viable for serious coding work. Our benchmark also found strong value from GLM routes:

Arm	Pass rate	Adjusted known cost	Cost per clean success	Unclean spend
router-glm-5.1	60.0%	\$20.30	\$0.58	\$4.10
router-glm-5.2	63.3%	\$25.32	\$0.72	\$8.23
router-gpt-5.5	73.3%	\$183.96	\$4.38	\$47.93

GPT-5.5 produced the strongest raw result in our Phase 3 run, but GLM 5.1 and GLM 5.2 were far more cost-efficient. This supports a tiered routing policy: reserve premium models for tasks where the higher success probability is worth the cost, and use cheaper frontier-adjacent models for routine work or budget-constrained sweeps.

Finding 3: token price is not the same as task cost

Databricks explicitly warns that token price can be a poor indicator of real end-to-end task cost. Our data confirms that theme, but through a different lens.

Examples from our Phase 3 results:

Arm	Pass rate	Adjusted known cost	Failure/incomplete spend	Unclean token share
router-anthropic-haiku-sanitized	41.7%	\$83.51	\$54.82	65.7%
router-deepseek-flash	66.7%	\$56.80	\$13.88	61.8%
router-gemini-flash	21.7%	\$43.53	\$24.26	56.7%

Haiku is the clearest cautionary result: a cheaper model class did not translate into cheap successful outcomes in this run. It had low pass rate and high failure/incomplete spend.

DeepSeek Flash shows the opposite nuance: it consumed many unclean tokens, but direct dollar waste was contained by pricing. That means token-volume efficiency and dollar efficiency should be tracked separately.

Finding 4: harness effects matter, but our design isolates backend route effects

Databricks found that harness choice can dramatically change cost and quality, including cases where simpler harnesses used much less context and reduced cost.

Our Phase 3 design intentionally fixed Claude Code as the harness. That makes the comparison less broad than Databricks, but cleaner for one question: how do backend/model routes behave when the agent harness is held constant?

The limitation is that our results should not be read as universal model rankings. A model that looks inefficient under Claude Code may perform differently with a different harness, context-management policy, timeout policy, or tool-use strategy.

The advantage is that backend differences are easier to interpret because the agent wrapper is not changing at the same time.

Finding 5: our added contribution is measuring nonproductive spend

The Databricks article emphasizes task-level cost. Our Phase 3 accounting adds a more explicit decomposition of nonproductive spend.

Across 15 valid full-suite arms and 900 attempts:

Metric	Value
Recorded cost	\$820.77
Adjusted known cost	\$972.17
Known accounting gap	\$151.40
Failure/incomplete spend	\$335.76
Unclean spend	\$364.29
Failure/incomplete token volume	213,482,015 tokens
Non-clean token volume	218,979,987 tokens

This matters because failed attempts are not free. They consume money, tokens, runtime, and review attention. A benchmark that reports only pass rate and successful-task cost can understate the operational burden of a route that fails expensively.

Finding 6: qualitative failure modes change the interpretation

Our qualitative review found that models fail in different ways:

- clean wrong answers

- exception failures after real token usage
- no-token/no-op signatures
- reward-1 rows with exception markers
- expensive failures where the model keeps working
- cheap-dollar but high-token failures

These modes have different remediation paths. Some point toward model capability, while others point toward route setup, harness behavior, provider reporting, timeout behavior, or ingestion quality.

This is why our benchmark separates:

- raw pass rate
- clean success
- exception-with-success-signal
- failure/incomplete spend
- unclean spend
- unresolved cost rows

Where the reports agree

1. A single default model is not optimal.
2. Cost/performance should be evaluated at the task or attempt level.
3. Token price alone is not enough to predict real cost.
4. Open or alternative-provider models can be operationally important.
5. Benchmarking should use real execution evidence and tests/verifiers rather than LLM-judge scoring.

Where our benchmark differs

1. Databricks uses internal production PR-derived tasks; we use Terminal-Bench/Harbor.
2. Databricks compares harnesses; we fixed Claude Code and compared backend routes.
3. Databricks emphasizes internal developer productivity; we emphasize benchmark reproducibility, artifact provenance, and provider cost accounting.
4. Our report adds an explicit adjusted-cost layer for missing-cost reconstruction.
5. Our report quantifies nonproductive spend and token-volume waste by outcome bucket.

Practical takeaway

The Databricks benchmark and our Phase 3 benchmark point in the same strategic direction: coding-agent selection should be data-driven, task-level, and route-aware.

For our Phase 3 data, the most important practical findings are:

- `router-gpt-5.5` had the highest raw pass rate, but at premium cost.
- `router-glm-5.1`, `router-glm-5.2`, `router-anthropic-fable-5`, and `router-deepseek-flash` formed economically important parts of the frontier.
- Recorded cost understated known benchmark spend by \$151.40.
- \$335.76 was spent on failed or incomplete attempts.
- 218.98M input+output tokens were consumed by non-clean outcomes.

- Future benchmark dashboards should show nonproductive spend and token waste alongside pass rate.

Recommended follow-up

The next benchmark phase should combine the two perspectives:

1. Keep the adjusted-cost and unclean-spend accounting layer.
2. Add harness comparisons, not only backend comparisons.
3. Preserve trajectory review so failures can be categorized by operational mode.
4. Report cost per clean success, failure/incomplete spend, and unclean token share as first-class metrics.
5. Use routing policies rather than a single default model.